Documentación del Proyecto Compilador

Etapa 6: Optimizaciones de Código y Testing

1 División del Trabajo

El desarrollo de la sexta y última etapa del compilador, centrada en el diseño e implementación del módulo de optimizaciones sobre el Árbol de Sintaxis Abstracta (`optimizations`), su integración con el gestor de compilación y la interfaz de línea de comandos, así como la elaboración de pruebas unitarias automatizadas, se realizó de manera colaborativa distribuyendo las responsabilidades de la siguiente forma:

- **Diseño e implementación de Optimizaciones (`optimizations.c/.h`):** Gribaudo Re Francisco, Francisco Andreani y Valentín Pastre Thuer.
- **Integración con el gestor del compilador y CLI (`compiler-manager.c/.h`, `cli.c`, `constants.h`):** Gribaudo Re Francisco, Francisco Andreani y Valentín Pastre Thuer.
- **Elaboración de pruebas unitarias y de integración (`test_optimizations.c`):** Gribaudo Re Francisco.

2 Decisiones de Diseño, Suposiciones y Aclaraciones

Habiendo completado en las etapas previas la cadena completa de compilación desde el análisis léxico hasta la generación de código ensamblador x86-64, el objetivo de esta última etapa consistió en incorporar técnicas de optimización que reduzcan el costo computacional y el tamaño del código generado sin alterar la semántica del programa original.

- **Optimización directa sobre el AST en Post-Orden:** Se decidió aplicar tanto el plegado de constantes (*Constant Folding*) como la eliminación de código muerto/inalcanzable (*Dead Code Elimination*) directamente sobre el Árbol de Sintaxis Abstracta (`syntaxTree`) inmediatamente después de la validación semántica y antes de la generación de Código Intermedio de Tres Direcciones (`codinter`). De esta manera, toda simplificación o poda realizada en el árbol impacta de forma transparente tanto en el archivo de código intermedio (`.ci`) como en el código ensamblador final (`.ass`), evitando la emisión innecesaria de variables temporales, etiquetas y saltos.
- **Secuenciación de Pasadas (*Constant Folding* previo a *Dead Code*):** En `codinterStage()` se dispuso que `constantOptimizations` se ejecute antes que `deadCodeOptimizations`. Esta decisión permite que, cuando se invoca el compilador con el nivel `-opt all`, las expresiones condicionales compuestas por constantes booleanas (por ejemplo, `if (true && false)` o `while (!true)`) sean primero reducidas a un único literal booleano (`false`) y posteriormente eliminadas por completo en la pasada de código inalcanzable.

- **Gestión de Memoria Dinámica en la Poda del AST (`freeAST`):** A diferencia de una simple desconexión de punteros, se implementó la función recursiva `freeAST(ASTNode *root)` para liberar explícitamente la memoria de las ramas descartadas durante la eliminación de código inalcanzable (tanto en bloques condicionales falsos como en sentencias posteriores a un `return`), previniendo fugas de memoria (*memory leaks*).
- **Detección Temprana de División y Módulo por Cero:** Durante el plegado de constantes aritméticas, si se detecta en tiempo de compilación una operación de división (`divOp`) o módulo (`modOp`) cuyo divisor literal es `0` (o `0.0`), el optimizador informa el error por salida estándar y aborta la compilación con `exit(1)`, evitando generar código inválido o excepciones de punto flotante en tiempo de ejecución.

3 Diseño y Decisiones Clave: El Módulo de Optimizaciones

El desarrollo central de esta etapa se encuentra en `src/optimizations.c` y `header/optimizations.h`, estructurado en torno a dos grandes algoritmos de transformación del AST:

- **Plegado de Constantes (`constantOptimizations`, `constantOptimizationsHelper`, `foldConstantOperation`, `createLiteralSymbol`):**
  - La función de entrada `constantOptimizations(enum optimizations optLevel)` verifica si el nivel solicitado corresponde a `constants` o `all`, invocando en tal caso a `constantOptimizationsHelper(syntaxTree)`.
  - `constantOptimizationsHelper` realiza un recorrido recursivo en **post-orden**, optimizando primero los subárboles izquierdo (`node->left`) y derecho (`node->right`). Esta estrategia de abajo hacia arriba (*bottom-up*) garantiza que expresiones anidadas complejas como `(2 + 3) * 4 - 6 / 2` propaguen sus resultados parciales desde las hojas hacia la raíz en una sola pasada.
  - Si tras optimizar ambos hijos se comprueba que tanto el operando izquierdo como el derecho son hojas literales (`intLiteral`, `floatLiteral` o `boolLiteral`), o bien que se trata de un operador unario (`negationOp` o `substractOp`) aplicado sobre un literal, se delega la evaluación a `foldConstantOperation`.
  - `foldConstantOperation` extrae los valores almacenados en la unión `value` de los símbolos operandos (realizando la promoción a `float` cuando interviene algún `tFloat`) y resuelve en tiempo de compilación las operaciones aritméticas (`addOp`, `substractOp`, `multOp`, `divOp`, `modOp`), relacionales (`lessOp`, `greaterOp`, `equalsOp`) y lógicas (`andOp`, `orOp`, `negationOp`).
  - Finalmente, `createLiteralSymbol` instancia un nuevo `Symbol` con el tipo y valor resultante (`hasValue = 1`), y se retorna un nuevo nodo hoja mediante `makeLeaf(resultSymbol)` que reemplaza al subárbol de la operación original.
- **Eliminación de Código Muerto e Inalcanzable (`deadCodeOptimizations`, `optimizeUnreachableCode`, `freeAST`):**
  - La función `deadCodeOptimizations(enum optimizations optLevel)` se activa cuando el usuario especifica `dead_code` o `all`, actualizando la raíz del árbol mediante `optimizeUnreachableCode(syntaxTree)`.

- `optimizeUnreachableCode` recorre recursivamente el AST en post-orden e interviene sobre dos patrones estructurales de código inalcanzable:
- **1. Sentencias posteriores a un `return` (`statementList`):** En la gramática del proyecto, las secuencias de sentencias se representan como listas enlazadas por derecha de nodos `statementList`, donde `node->left` aloja la sentencia actual y `node->right` apunta al resto de las sentencias del bloque. Cuando `optimizeUnreachableCode` detecta que `node->left` es un nodo `returnNode`, invoca a `freeAST(node->right)` y asigna `node->right = NULL`, podando todas las instrucciones inalcanzables que suceden al retorno dentro de ese bloque.
- **2. Estructuras de control con condición constante falsa (`ifNode` y `whileNode`):** Cuando el nodo analizado es una sentencia condicional `if` o un ciclo `while`, se inspecciona su expresión de condición en `node->left`. Si la condición es un literal booleano (`boolLiteral`) cuyo valor de verdad es falso (`value.boolValue == 0`), el bloque nunca se ejecutará; por consiguiente, se libera la estructura completa mediante `freeAST(node)` y se retorna `NULL`.

4 Integración con el Resto del Compilador

Para exponer y coordinar las distintas pasadas de optimización se actualizaron los siguientes componentes del compilador:

- **Definición de Niveles de Optimización (`constants.h`):** Se definió el enumerado `enum optimizations { none, constants, dead_code, memory, all }` para representar internamente el modo de optimización seleccionado.
- **Interfaz de Línea de Comandos (`cli.c/.h`):** En `handleCliInput()` se procesa el argumento posterior a la bandera `-opt`, y en `returnOptimization()` se traduce la cadena ingresada por consola (`"none"`, `"constants"`, `"dead-code"`, `"memory"` o `"all"`) al valor correspondiente del enumerado `enum optimizations`, estableciendo `none` como comportamiento por defecto cuando se omite la bandera.
- **Gestor de Compilación (`compiler-manager.c/.h`):** Se incluyó `optimizations.h` y se integraron las llamadas a `constantOptimizations(optimizationLevel)` y `deadCodeOptimizations(optimizationLevel)` dentro de `codinterStage()`, justo después de ejecutar `semanticStage()` y antes de `initCodinterGenerator()` y `generateCodinterFromAST(syntaxTree)`. Gracias al diseño en cascada del gestor (`assemblyStage` invoca a `codinterStage`), las optimizaciones se aplican tanto al generar código intermedio (`-target codinter`) como al compilar hasta código ensamblador (`-target assembly`).

5 Instrucciones de Compilación y Ejecución de Pruebas

El proyecto mantiene la integración con **Ceedling** (Unity + CMock) para la ejecución de pruebas unitarias y la compilación del ejecutable `c-tds`.

5.1. Ejecución de Tests Unitarios con Ceedling

En esta etapa se desarrolló la suite `test/test_optimizations.c`, la cual verifica directamente sobre el AST en memoria el plegado de expresiones aritméticas enteras combinadas, el plegado de expresiones booleanas con negación, el plegado parcial en expresiones que combinan variables con subexpresiones constantes, la poda de sentencias e iteraciones posteriores a un `return`, y la eliminación de bloques `if (false)` y `while (false)`:

- **Correr tests del Módulo de Optimizaciones:**

  ```
  ./ceedling test:optimizations
  ```

- **Correr la batería completa de tests del proyecto:**

  ```
  ./ceedling test:all
  ```

5.2. Construcción y Uso del Compilador con Banderas de Optimización

Para compilar la versión de lanzamiento (*Release*) y ejecutar el compilador aplicando las distintas optimizaciones disponibles sobre un archivo fuente `.ctds`:

```
./compile.sh

# Aplicar únicamente plegado de constantes:
./build/artifacts/release/c-tds -target assembly -opt constants -o salida prueba.ctds

# Aplicar únicamente eliminación de código muerto/inalcanzable:
./build/artifacts/release/c-tds -target assembly -opt dead-code -o salida prueba.ctds

# Aplicar todas las optimizaciones disponibles:
./build/artifacts/release/c-tds -target assembly -opt all -o salida prueba.ctds
```

6 Problemas Conocidos y Testing

Durante el desarrollo y testeo de esta sexta etapa se identificaron y analizaron los siguientes aspectos técnicos:

- **Plegado Parcial en Expresiones Mixtas:** Al evaluar expresiones donde intervienen tanto variables como constantes (por ejemplo, `x = x + (3 * 4);`), fue fundamental asegurar que `constantOptimizationsHelper` conservara intacto el nodo padre (`addOp`) cuando uno de sus hijos es un identificador (`variable`), plegando únicamente el subárbol derecho (`3 * 4` a `12`). Este comportamiento fue validado mediante el test unitario `test_constant_folding_parcial_con_variables`.

- **Tipado de Operaciones Relacionales Plegadas y Resta Unaria:** En `foldConstantOperation`, la bandera `isBool` se determina inicialmente verificando si alguno de los operandos de entrada es de tipo `tBoolean`. Como consecuencia, mientras que las operaciones lógicas (`&&`, `||`, `!`) generan un nodo `boolLiteral`, las comparaciones relacionales entre literales numéricos (<, >, ==) se pliegan actualmente en un `intLiteral` (`1` o `0`). Asimismo, dado que el parser representa la resta unaria

(`-expr`) ubicando el operando en `node->left` y dejando `node->right` en `NULL`, la creación del símbolo comodín derecho en `constantOptimizationsHelper` evalúa `leftInt - 0` en lugar de invertir el signo. Ambos casos fueron identificados durante las pruebas de caja blanca sobre el AST como oportunidades de mejora puntual en `foldConstantOperation`.

- **Liberación Segura de Memoria entre Casos de Prueba:** Para evitar que los árboles sintácticos generados en cada test unitario de `test_optimizations.c` quedaran retenidos en memoria al finalizar la prueba, se incorporó la llamada a `freeAST(syntaxTree)` dentro de la función `tearDown()` de Unity, verificando a la vez que la poda realizada por `optimizeUnreachableCode` no dejara punteros colgantes (*dangling pointers*) en el AST principal.